

HIKNet: Head Impact Detection from Kinematic Data

Guru R Bharadwaj (PES1UG24CS177) - UE24CS352A Machine Learning mini project

1. Problem and data

Instrumented mouthguards record linear acceleration and angular velocity whenever a 10g threshold is crossed, but many recordings are not head impacts (chewing, biting, dropping the mouthguard). The task is binary classification of each 6 x 199 recording (lin acc x/y/z, ang vel x/y/z, 1000 Hz, ~0.2 s) into real impact vs false positive. This is a PyTorch port and extension of HIKNet (Fantoni, Gaudio, Ling, CS229, Stanford).

Important: the original lab data was never released, so all neural network results here come from a synthetic mock dataset with the same shape and class balance: 527 samples, 264 impacts and 263 false positives. Impacts are simulated as damped 18-32 Hz pulses, false positives ring faster with more 80-200 Hz energy, and 16 samples are drawn from the opposite class so the task is not trivially separable. The numbers below therefore say nothing about real-world performance; they only check that the pipeline and models behave sensibly.

2. Method

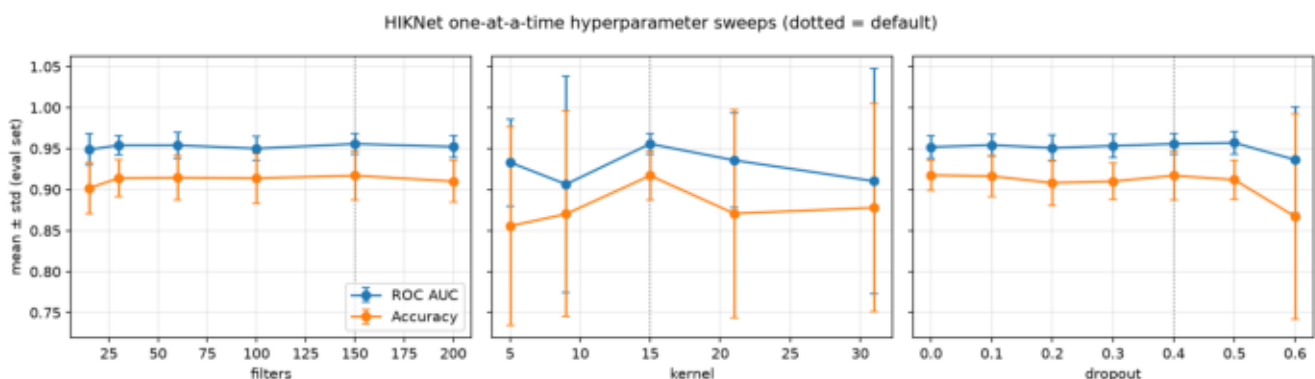
SVM baseline. An RBF SVM (standardized inputs, default C and gamma) with greedy forward feature selection by 10-fold CV ROC AUC, up to 10 features. On the public Wu et al. (2017) feature set (411 features, 387 collegiate samples, 32 youth test samples) it reaches 90.2% accuracy, 78.2% sensitivity and ROC AUC 0.978 in 10-fold CV, and 96.9% accuracy on the youth test set. For the mock data the same procedure runs on time-domain and PSD features computed from the raw traces.

HIKNet. Two 1D conv blocks (Conv1d with 150 filters and kernel 15, ReLU, max-pool 2, dropout) over the 6 channels, then the (time x filter) map is treated as a one-channel image and passed through a 2D conv (150 filters, 3 x 15 kernel, stride 3 along filters), ReLU, a pooling head and a single logit. Trained with Adam (lr 1e-3, batch 32) and binary cross-entropy, with early stopping (patience 5) on validation loss, restoring the best epoch. Inputs are standardized per channel.

RecursiveNet. The second architecture from the original code: a fully 2D CNN on the 6 x 199 signal image with 3 x 3 convolutions (32 and 64 filters), dropout 0.25, two U-Net-style skip concatenations, and a 256-unit dense layer before the output. It has far more parameters than HIKNet because of the large flattened dense layer.

3. Tuning

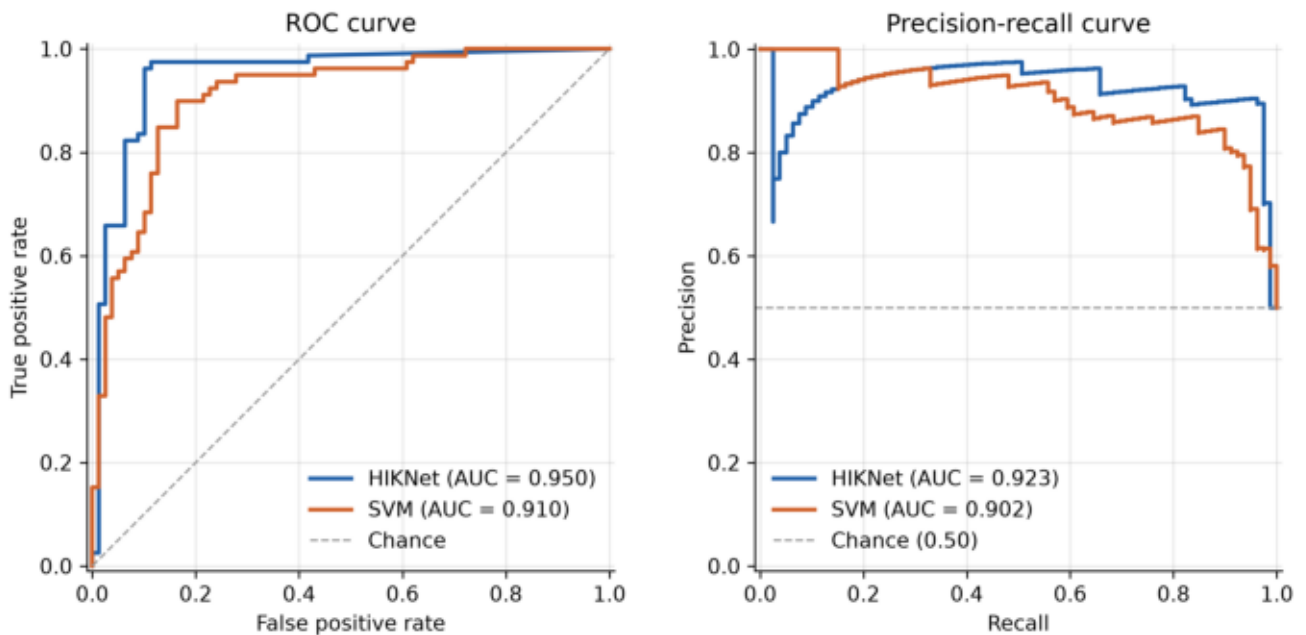
Each configuration was trained 10 times with different seeds (30 epochs max) on the 70/30 split and averaged. Output head: global average pooling 91.7% accuracy (ROC AUC 0.956), global max pooling 89.4%, flatten + dense only 68.4%, so GAP was kept. Filters (15-200) barely matter (90.2-91.7%). Kernel width matters most: 15 gives 91.7% vs 85.6% for 5 and 87.8% for 31. Dropout is flat from 0 to 0.5 and drops to 86.7% at 0.6. The best setting was filters 150, kernel 15, dropout 0.5, within noise of the default (dropout 0.4), which was kept.



4. Results

Final comparison on the held-out 30% of the mock data ($n = 158$, seed 0). The SVM uses features selected on the training part only. HIKNet stopped at epoch 12. RecursiveNet vs HIKNet is reported separately as 10-fold CV means (each fold's held-out part used for early stopping and scoring, as in the original code).

Model	Accuracy	Precision	Specificity	Sensitivity	ROC AUC	PR AUC
HIKNet (split)	0.899	0.899	0.899	0.899	0.950	0.923
SVM (split)	0.759	0.873	0.911	0.608	0.910	0.902
HIKNet (10-fold)	0.936	0.915	0.909	0.962	0.967	0.961
RecursiveNet (10-fold)	0.820	0.758	0.848	0.793	0.882	0.902



On the split HIKNet beats the SVM in accuracy (89.9% vs 75.9%) and especially sensitivity (89.9% vs 60.8%); the SVM is slightly more specific. ROC AUC is 0.950 vs 0.910. The SVM's selected features are mostly 20-30 Hz PSD bins, the band where the mock impacts were placed. In 10-fold CV, RecursiveNet reaches 82.0% accuracy vs 93.6% for HIKNet, with ROC AUC 0.882 vs 0.967; HIKNet gets there with far fewer parameters.

5. Limitations

(1) Mock data: everything except the Wu et al. SVM numbers comes from synthetic signals whose class difference is a designed frequency shift, so absolute numbers are not comparable to the original report or to real deployment. (2) The evaluation split is also used for early stopping (as in the original code), so neural network scores are optimistically biased; a separate validation set would remove this. (3) The final table is a single 70/30 split with one seed ($n = 158$); the 10-repeat tuning spread (std about 3 points of accuracy) shows single-split differences of a few points are within noise. (4) The SVM runs with default hyperparameters.

6. Conclusion

The PyTorch port of HIKNet trains reliably end to end: global average pooling is clearly the right head, kernel width is the most sensitive hyperparameter, and filter count and moderate dropout matter little. On the mock data HIKNet outperforms a feature-selected SVM, mainly by catching more true impacts. Real conclusions need the lab recordings; the code is ready to rerun unchanged once data/data.mat and data/labels.mat are available.